

Stage 1: MuJoCo, Robot State, and PD control

Part 1: pup/sim/pd.py

equation 1: Policy Action $\rightarrow$ Desired Joint Position

$$q_{\text{target}} = q_{\text{default}} + S q A$$

desired joint angle
- where we want the joint to move (in radians)

default joint angle

action scale
- 0.5 rad for this project
- determines how far the neural net is allowed to move away from the default pose

neural network action between 1 and -1

"For this knee, move X radians away from the default to reach our target position"

$A = 0$ indicates $q_{\text{target}} = q_{\text{default}}$

equation 2: Desired joint position $\rightarrow$ motor torque

PD controller:
$$\tau = k_p(q_{\text{target}} - q) + k_d(\dot{q}_{\text{target}} - \dot{q})$$

τ = motor torque is the output (Newton meters)

position error term $k_p(q_{\text{target}} - q)$

k_p = proportional gain/"stiffness"

q_{target} = where we want the joint

q = where the joint actually is

$q_{\text{target}} - q$ = position error

farther away the joint is,

the more the controller

pushes towards the target

(think of it as a stiffer spring)

velocity error term $k_d(\dot{q}_{\text{target}} - \dot{q})$ dot means rate of change

k_d = derivative damping gain

$\dot{q}$ = actual angular velocity

$\dot{q}_{\text{target}}$ = desired angular velocity

desired velocity usually zero

for this project

$$k_d(0 - \dot{q}) = -\dot{q}k_d$$

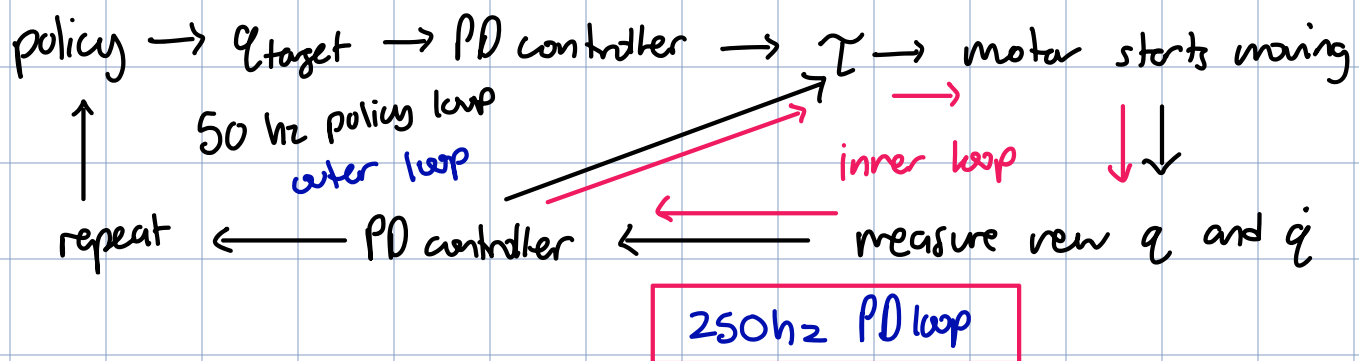
shock absorber, dampens

rapid movement

k_d opposes motion

RL policy decides what motion to request

PD controller decides how to physically track it



Inner PD loop happens 5 times for every policy loop

Part 2: pup/sim/standup.py

goal is to interpolate (create poses) between crouch and hane over specified time period (1 second)

- can't command motors to instantly move to desired position (ie no teleporting)

if robot at perfectly desired position with 0 velocity

and 0 torque, no force except gravity/normal

so legs slightly sag → position error created → Torque

applied to counter balance gravitational force and reach

desired position

- 1 Set up simulation
- 2 Get starting and ending joint positions
- 3 Create PD controller
- 4 Calculate where we want the joints right now
- 5 Measure where the robot actually is
- 6 Get error and turn it into motor torque
- 7 Feed torques to MuJoCo and advance physics
- 8 Check if pup fell over with trunk quaternion
- 9 Return scorecard for script with final height, max roll, max pitch, and fell fields

